Отчет по лабораторной работе №6

Основы информационной безопасности

Кузьмин Егор Витальевич, НКАбд-01-23

Содержание

1 Цель работы 5

2 Теоретическое введение 6

3 Выполнение лабораторной работы 8

4 Выводы 18

Список литературы 19

Список иллюстраций

3.1 проверка режима работы SELinux 8
3.2 Проверка работы Apache 9
3.3 Контекст безопасности Apache 9
3.4 Состояние переключателей SELinux 10
3.5 Статистика по политике 10
3.6 Типы поддиректорий 11
3.7 Типы файлов 11
3.8 Создание файла 12
3.9 Контекст файла 12
3.10 Изучение справки по команде 13
3.11 Изменение контекста 14
3.12 Попытка прочесть лог-файл 14
3.13 Проверка лог-файлов 15
3.14 Проверка лог-файлов 15
3.15 Проверка портов 16
3.16 Перезапуск сервера 16
3.17 Удаление файла 17

Список таблиц

1 Цель работы

Развить навыки администрирования ОС Linux. Получить первое практическое знакомство с технологией SELinux1. Проверить работу SELinx на практике совместно с веб-сервером Apache.

2 Теоретическое введение

1. **SELinux (Security-Enhanced Linux)** обеспечивает усиление защиты путем внесения изменений как на уровне ядра, так и на уровне пространства пользователя, что превращает ее в действительно «непробиваемую» операционную систему. Впервые эта система появилась в четвертой версии CentOS, а в 5 и 6 версии реализация была существенно дополнена и улучшена.

SELinux имеет три основных режим работы:

- Enforcing: режим по умолчанию. При выборе этого режима все действия, которые каким-то образом нарушают текущую политику безопасности, будут блокироваться, а попытка нарушения будет зафиксирована в журнале.
- Permissive: в случае использования этого режима, информация о всех действиях, которые нарушают текущую политику безопасности, будут зафиксированы в журнале, но сами действия не будут заблокированы.
- Disabled: полное отключение системы принудительного контроля доступа.

Политика SELinux определяет доступ пользователей к ролям, доступ ролей к доменам и доступ доменов к типам. Контекст безопасности — все атрибуты SELinux — роли, типы и домены. Более подробно см. в [**f?**].

2. **Apache** — это свободное программное обеспечение, с помощью которого можно создать веб-сервер. Данный продукт возник как доработанная версия другого HTTP-клиента от национального центра суперкомпьютерных приложений (NCSA).

Для чего нужен Apache сервер:

- чтобы открывать динамические PHP-страницы,
- для распределения поступающей на сервер нагрузки,
- для обеспечения отказоустойчивости сервера,
- чтобы потренироваться в настройке сервера и запуске PHP-скриптов.

Apache является кроссплатформенным ПО и поддерживает такие операционные системы, как Linux, BSD, MacOS, Microsoft, BeOS и другие.

3 Выполнение лабораторной работы

Вхожу в систему под своей учетной записью. SELinux работает в режиме enforcing политики targeted, можно проверить с помощью команд getenforce и sestatus (рис. 1).

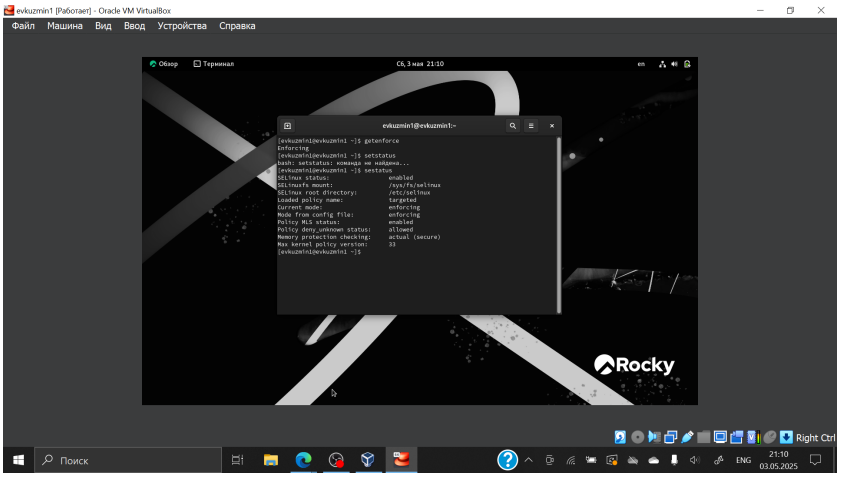

Рис. 3.1: проверка режима работы SELinux

Запускаю сервер apache, далее обращаюсь с помощью браузера к веб-серверу, запущенному на компьютере, он работает, что видно из вывода команды `service httpd status` (рис. 2).

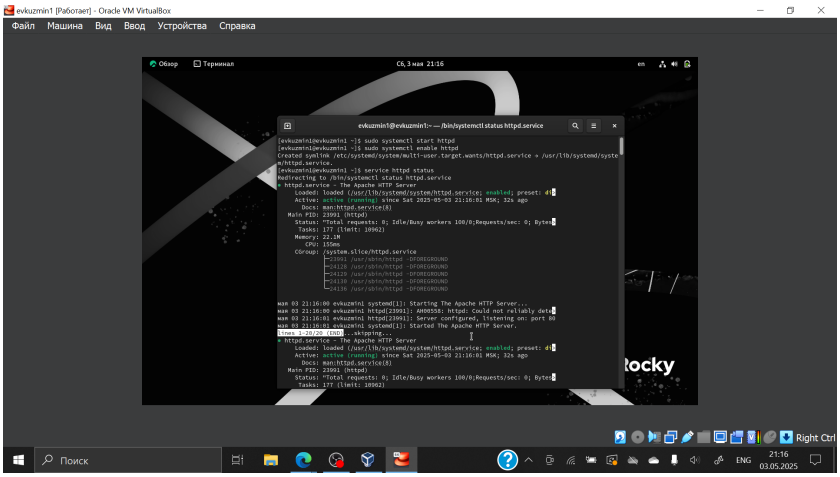

Рис. 3.2: Проверка работы Apache

С помощью команды `ps auxZ | grep httpd` нахожу веб-сервер Apache в списке процессов. (рис. 3).

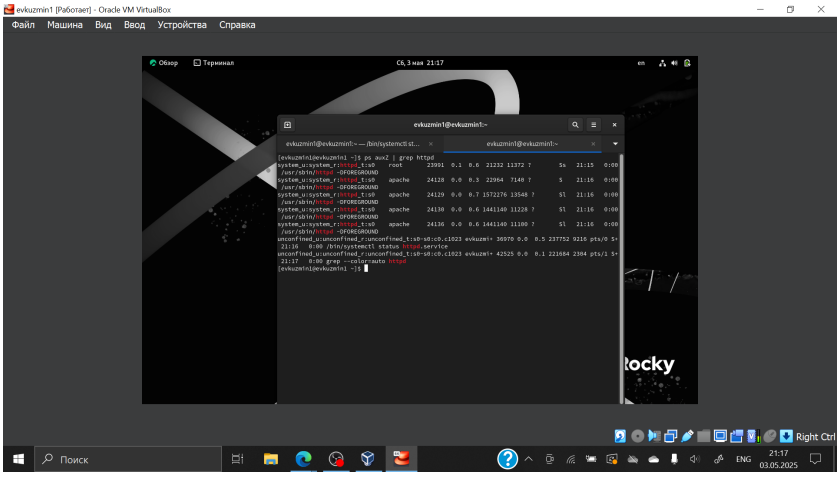

Рис. 3.3: Контекст безопасности Apache

Просматриваю текущее состояние переключателей SELinux для Apache с помощью команды `sestatus -bigrep httpd` (рис. 4).

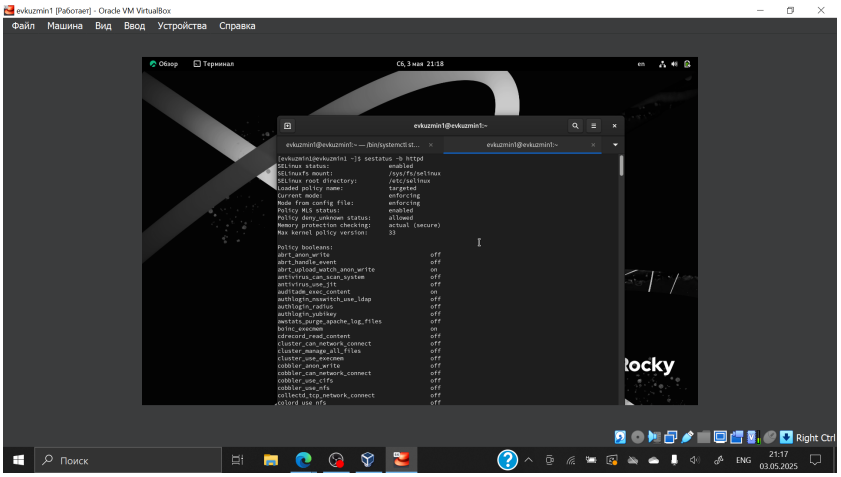

Рис. 3.4: Состояние переключателей SELinux

Смотрю статистику по политике с помощью команды `seinfo`. Множество пользователей - 8, ролей - 40, типов - 5169. (рис. 5).

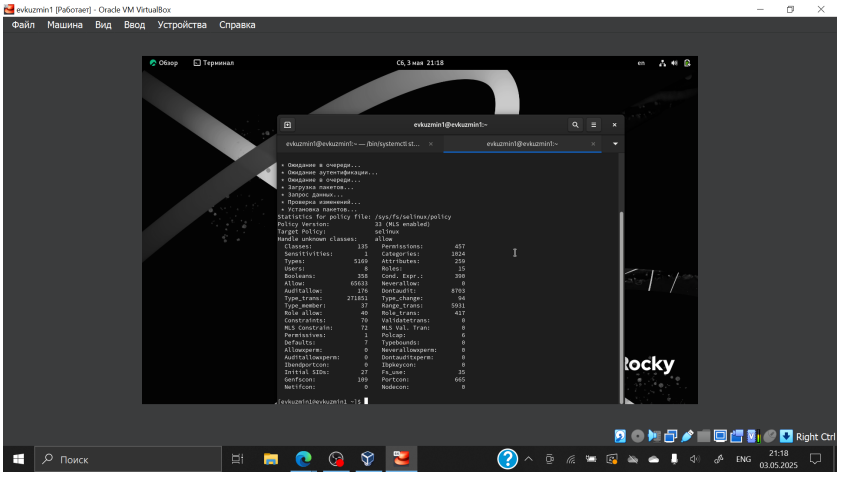

Рис. 3.5: Статистика по политике

Типы поддиректорий, находящихся в директории `/var/www`, с помощью команды `ls -lZ /var/www` следующие: владелец - root, права на изменения только у владельца. Файлов в директории нет (рис. 6).

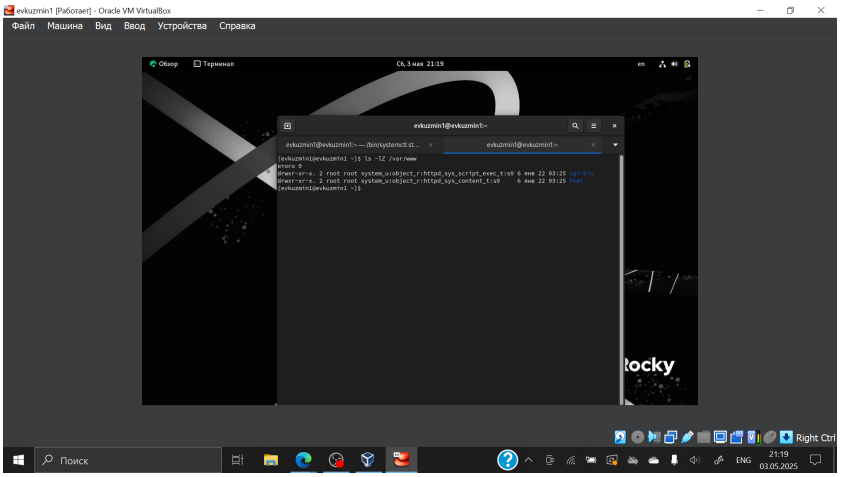

Рис. 3.6: Типы поддиректорий

В директории `/var/www/html` нет файлов. (рис. 7).

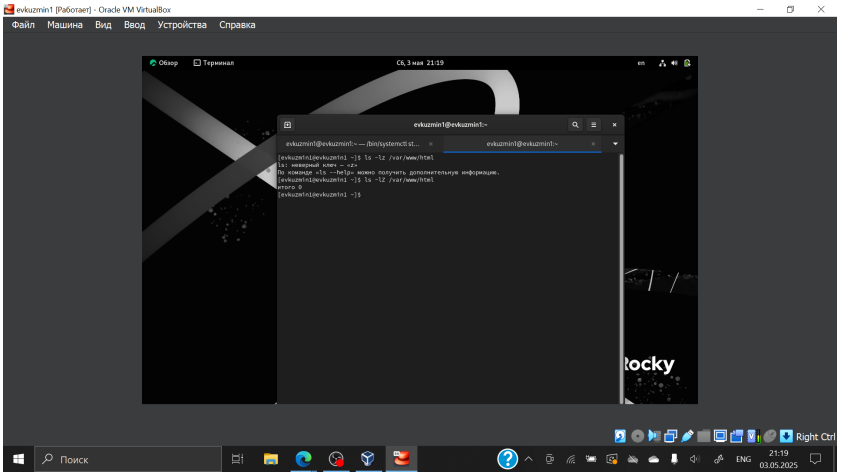

Рис. 3.7: Типы файлов

Создать файл может только суперпользователь, поэтому от его имени создаем файл touch.html со следующим содержанием:

```
<html>
<body>test</body>
</html>
```

(рис. 8).

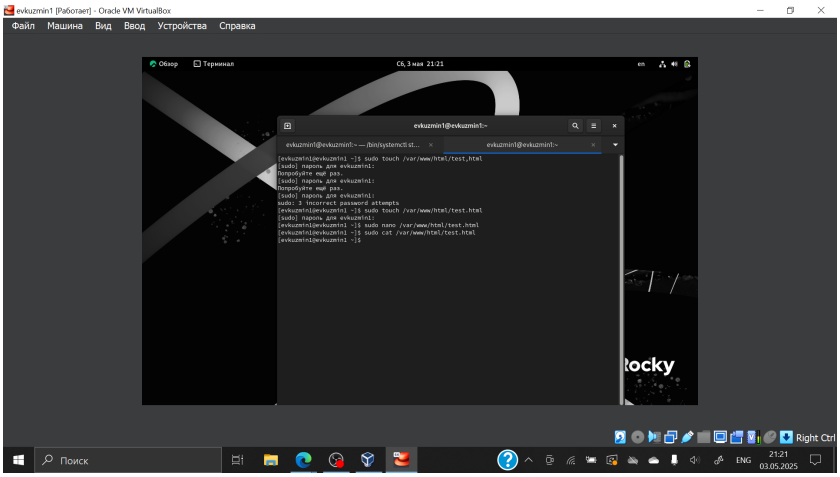

Рис. 3.8: Создание файла

Проверяю контекст созданного файла. По умолчанию это httpd_sys_content_t (рис. 9).

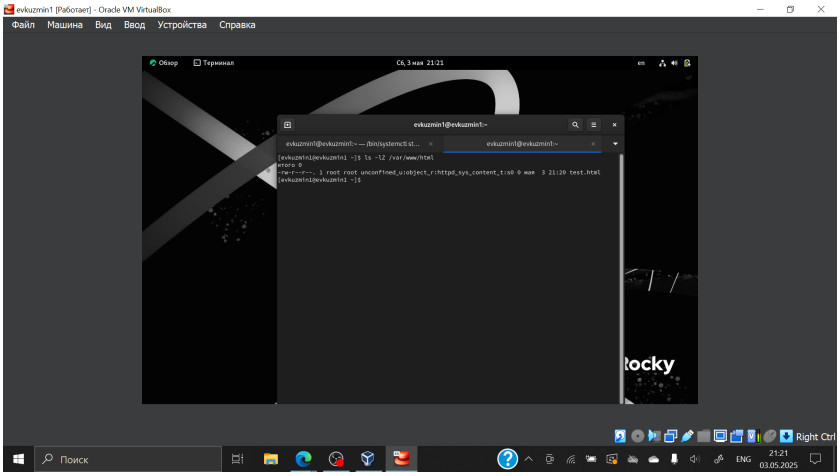

Рис. 3.9: Контекст файла

Изучаю справку man httpd_selinux. Рассмотрим полученный контекст детально. Так как по умолчанию пользователи CentOS являются свободными от типа (unconfined в переводе с англ. означает свободный), созданному нами файлу test.html был сопоставлен SELinux, пользователь unconfined_u. Это первая часть контекста. Далее политика ролевого разделения доступа RBAC используется процессами, но не файлами, поэтому роли не имеют никакого значения для файлов. Роль object_r используется по умолчанию для файлов на «постоянных» носителях

и на сетевых файловых системах. (В директории /proc файлы, относящиеся к процессам, могут иметь роль system_r. Если активна политика MLS, то могут использоваться и другие роли, например, secadm_r. Данный случай мы рассматривать не будем, как и предназначение :s0). Тип httpd_sys_content_t позволяет процессу httpd получить доступ к файлу. Благодаря наличию последнего типа мы получили доступ к файлу при обращении к нему через браузер. (рис. 10).

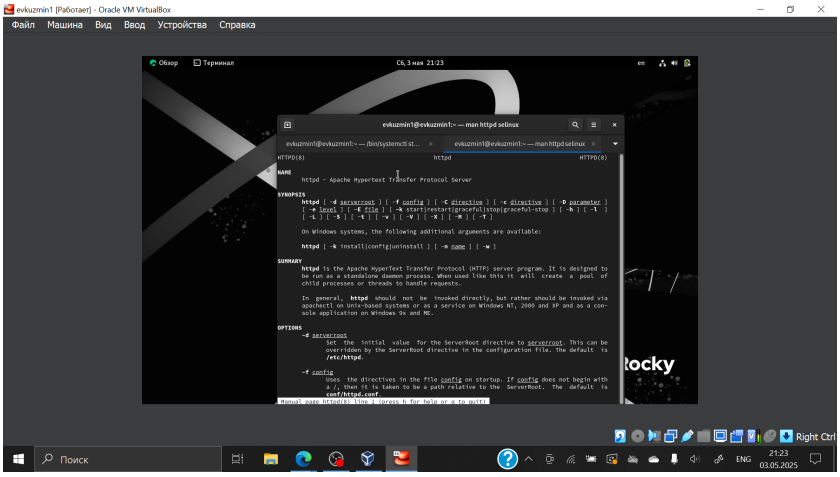

Рис. 3.10: Изучение справки по команде

Изменяю контекст файла `/var/www/html/test.html` с `httpd_sys_content_t` на любой другой, к которому процесс httpd не должен иметь доступа, например, на `samba_share_t`: `chcon -t samba_share_t /var/www/html/test.html ls -Z /var/www/html/test.html` Контекст действительно поменялся (рис. 11).

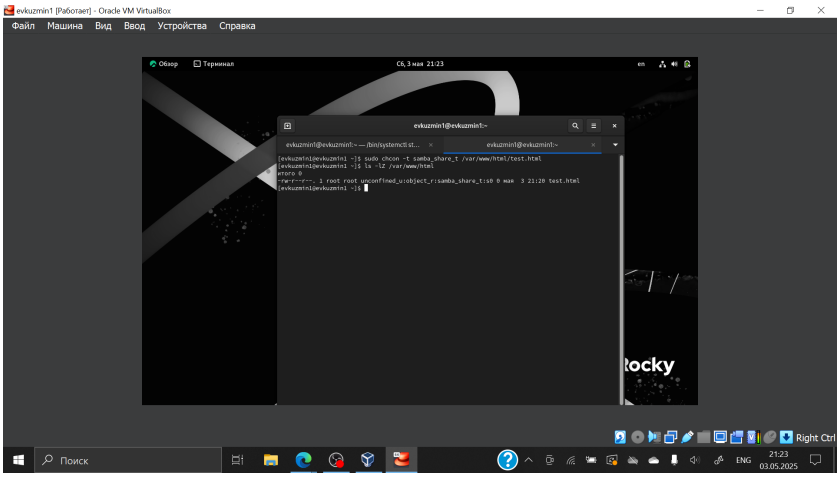

Рис. 3.11: Изменение контекста

Просматриваю log-файлы веб-сервера Apache и системный лог-файл: `tail /var/log/messages`. Если в системе окажутся запущенными процессы setroubleshootd и audtd, то вы также сможете увидеть ошибки, аналогичные указанным выше, в файле /var/log/audit/audit.log. (рис. 12).

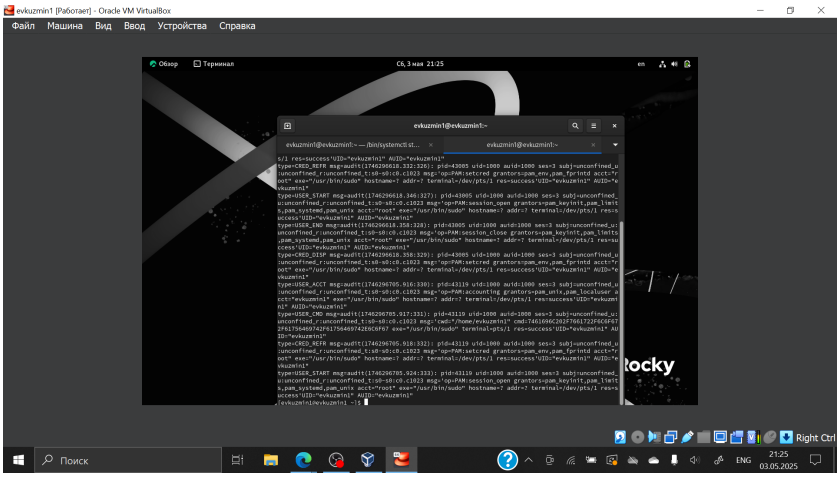

Рис. 3.12: Попытка прочесть лог-файл

Проанализируйте лог-файлы: `tail -nl /var/log/messages` (рис. 14).

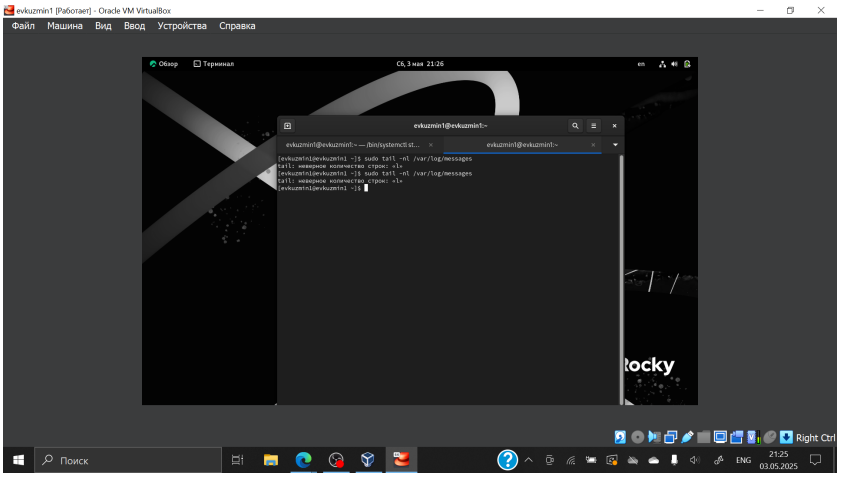

Рис. 3.13: Проверка лог-файлов

Просмотрите файлы `/var/log/http/error_log`, `/var/log/http/access_log` и `/var/log/audit/audit.log` и выясните, в каких файлах появились записи. Запись появилась в файлу error_log (рис. 14).

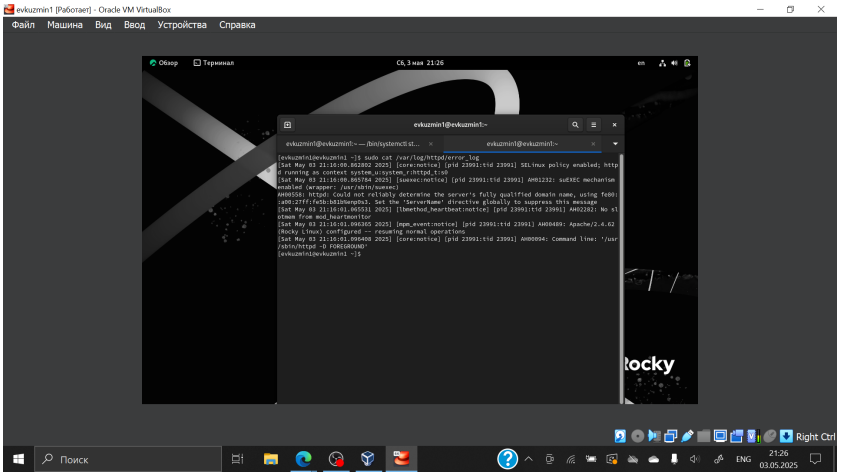

Рис. 3.14: Проверка лог-файлов

Выполняю команду `semanage port -a -t http_port_t -p tcp 81` После этого проверяю список портов командой `semanage port -l | grep http_port_t` Порт 81 появился в списке (рис. 15).

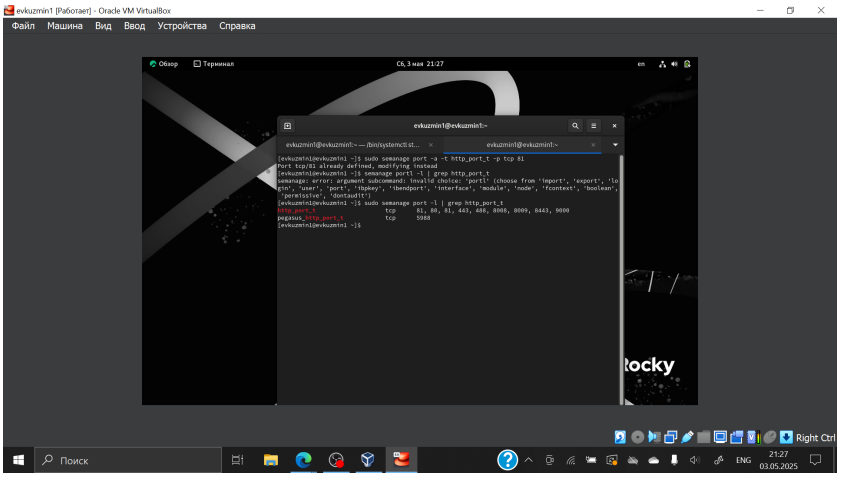

Рис. 3.15: Проверка портов

Перезапускаю сервер Apache (рис. 16).

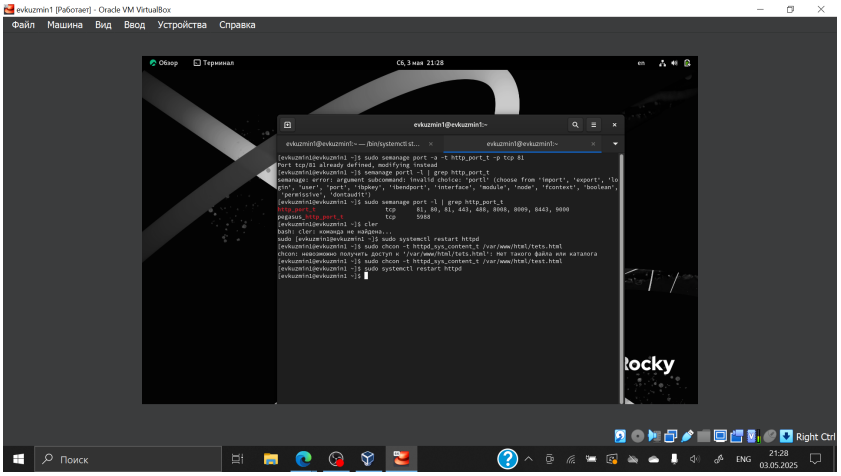

Рис. 3.16: Перезапуск сервера

Возвращаю в файле /etc/httpd/httpd.conf порт 80, вместо 81. Проверяю, что порт 81 удален, это правда.

Далее удаляю файл test.html, проверяю, что он удален (рис. 17).

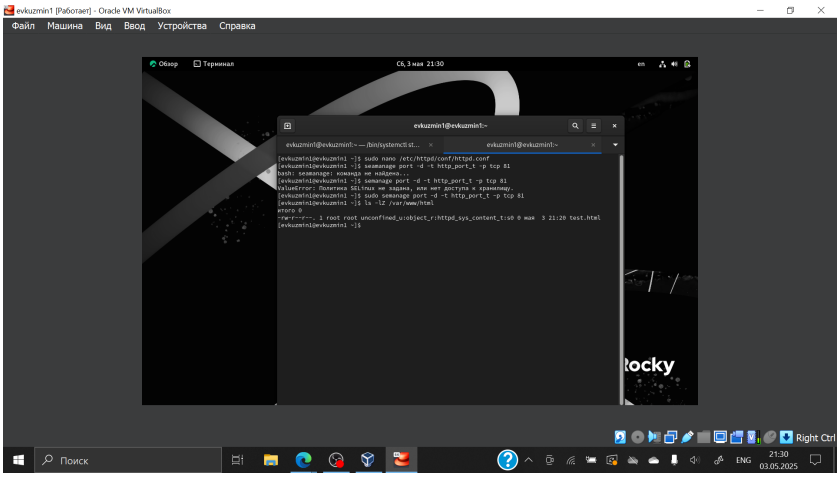

Рис. 3.17: Удаление файла

4 Выводы

В ходе выполнения данной лабораторной работы были развиты навыки администрирования ОС Linux, получено первое практическое знакомство с технологией SELinux и проверена работа SELinux на практике совместно с веб-сервером Apache.

Список литературы

:::